

Experiment 5: Simulation of QPSK Modulation and Demodulation through an AWGN Channel

Output Code

```
import numpy as np
import matplotlib.pyplot as plt
import cv2
from scipy.special import erfc

img = cv2.imread("/content/cameraman.png", cv2.IMREAD_GRAYSCALE)

M, N = img.shape
print("Image size:", M, "x", N)

print(img.max())
print(img.min())
x = img / 255

print(x.max())
print(x.min())

x_vec = x.flatten()
```

PCM Encoder (8-bit Quantization)

```
def pcm_encode(x, b):
    L = 2**b
    delta = 1/L

    k = np.floor(x/delta)
    k = np.clip(k, 0, L-1).astype(int)
```

```

xcap = (k + 0.5)*delta

Ps = np.mean(x**2)
Pn = np.mean((x - xcap)**2)
SQNRdB = 10*np.log10(Ps/Pn)

bitstream=""
for v in k:
    bitstream+=np.binary_repr(v,width=b)

b_tx = np.array(list(bitstream), dtype=np.uint8)

return k, xcap, b_tx, SQNRdB

b = 8
b_tx = pcm_encode(x_vec, b)

```

Channel Modelling – QPSK Modulation and AWGN

```

def qpsk_modulate(b):
    if len(b) % 2 != 0:
        b = np.append(b, 0)

    b = b.reshape(-1, 2)
    s = []

    for pair in b:
        if pair[0] == 0 and pair[1] == 0:
            s.append(1 + 1j)
        elif pair[0] == 0 and pair[1] == 1:
            s.append(-1 + 1j)
        elif pair[0] == 1 and pair[1] == 1:
            s.append(-1 - 1j)
        else:
            s.append(1 - 1j)

    return np.array(s)

def qpsk_demodulate(y):
    b_rx = []

```

```

for r in y:
    if r.real >= 0 and r.imag >= 0:
        b_rx.extend([0, 0])
    elif r.real < 0 and r.imag >= 0:
        b_rx.extend([0, 1])
    elif r.real < 0 and r.imag < 0:
        b_rx.extend([1, 1])
    else:
        b_rx.extend([1, 0])

return np.array(b_rx, dtype=np.uint8)
def awgn_qpsk(b_tx, EbNO_dB):
    Eb = 1
    s = qpsk_modulate(b_tx)

    EbNO = 10**(EbNO_dB/10)
    NO = Eb / EbNO
    sigma2 = NO / 2

    noise = np.sqrt(sigma2) * (np.random.randn(len(s)) +
                               1j*np.random.randn(len(s)))

    y = s + noise

    b_rx = qpsk_demodulate(y)
    b_rx = b_rx[:len(b_tx)]

    ber = np.mean(b_rx != b_tx)
    return y, b_rx, ber

```

PCM Decoder and BER Computation

```

def pcm_decode(b_rx, b, M, N):
    L = 2**b
    delta = 1 / L

    total_bits = (len(b_rx) // b) * b
    b_rx = b_rx[:total_bits]

    bits = b_rx.reshape(-1, b)

    k_hat = []

```

```

for row in bits:
    binary_string = ""
    for bit in row:
        binary_string += str(bit)

    index = int(binary_string, 2)
    k_hat.append(index)

k_hat = np.array(k_hat)

k_hat = np.clip(k_hat, 0, L-1)

xcap = (k_hat + 0.5) * delta

xcap_img = xcap.reshape(M, N)

return xcap_img

```

Main function

```

EbN0_dB = [0, 2, 4, 6, 8, 10]
BER_sim = []

for ebn0 in EbN0_dB:
    y, b_rx, ber = awgn_qpsk(b_tx, ebn0)
    BER_sim.append(ber)

    xcap_img = pcm_decode(b_rx, b, M, N)

    plt.figure(figsize=(10,4))
    plt.subplot(1,2,1)
    plt.imshow(xcap_img, cmap='gray')
    plt.title(f"Eb/N0 = {ebn0} dB, BER = {ber:.3e}")
    plt.axis('off')

    plt.subplot(1,2,2)
    plt.scatter(y.real[:4000], y.imag[:4000], s=2)
    plt.title(f"QPSK Scatter Plot (Eb/N0 = {ebn0} dB)")
    plt.xlabel("In-phase")
    plt.ylabel("Quadrature")
    plt.grid()

```

```

plt.show()

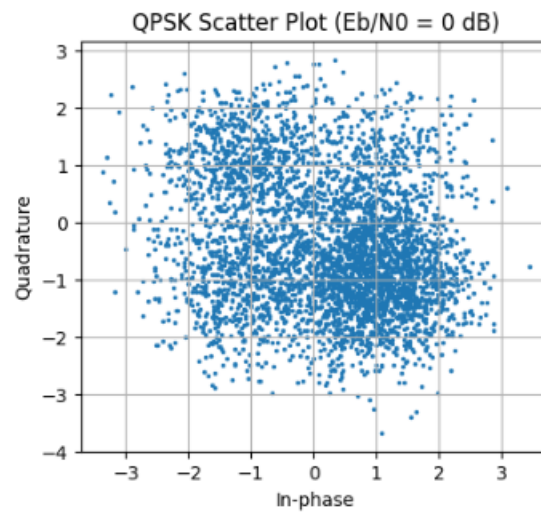
BER_theory = [0.5 * erfc(np.sqrt(10*(e/10))) for e in EbNO_dB]

plt.figure()
plt.semilogy(EbNO_dB, BER_sim, 'o-', label='Simulated')
plt.semilogy(EbNO_dB, BER_theory, '--', label='Theoretical')
plt.xlabel("Eb/NO (dB)")
plt.ylabel("BER")
plt.grid(True, which='both')
plt.legend()
plt.show()

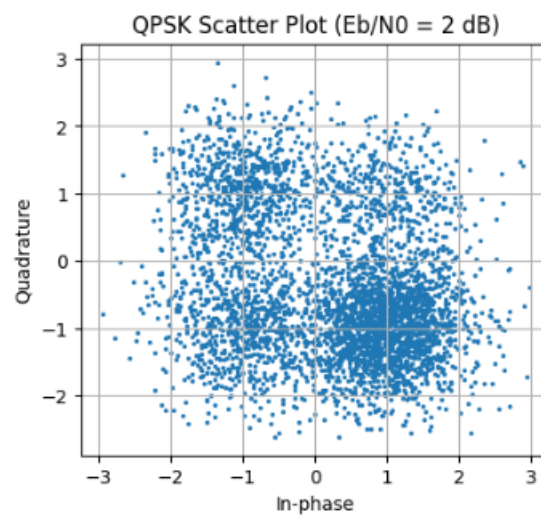
```

Reconstructed Images for Different E_b/N_0 Values

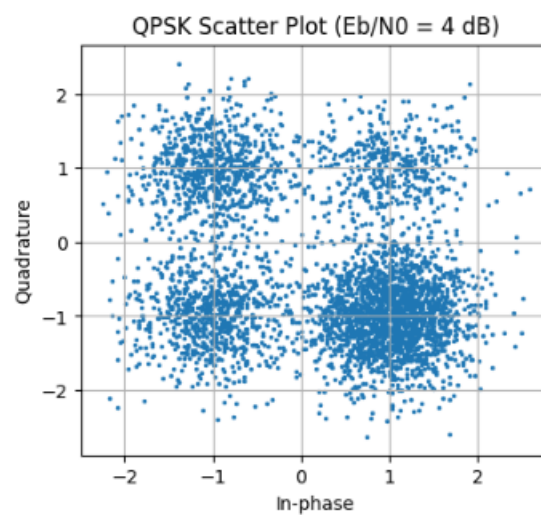
$E_b/N_0 = 0$ dB, BER = 7.804×10^{-2}



$E_b/N_0 = 2$ dB, BER = 3.763×10^{-2}



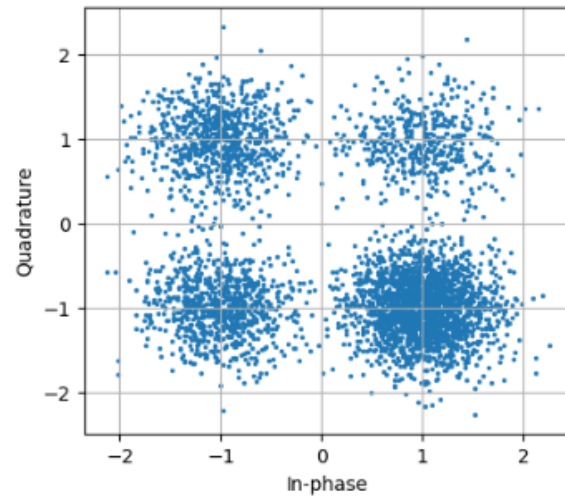
$E_b/N_0 = 4$ dB, BER = 1.250×10^{-2}



$E_b/N_0 = 6 \text{ dB}$, $\text{BER} = 2.270\text{e-}03$



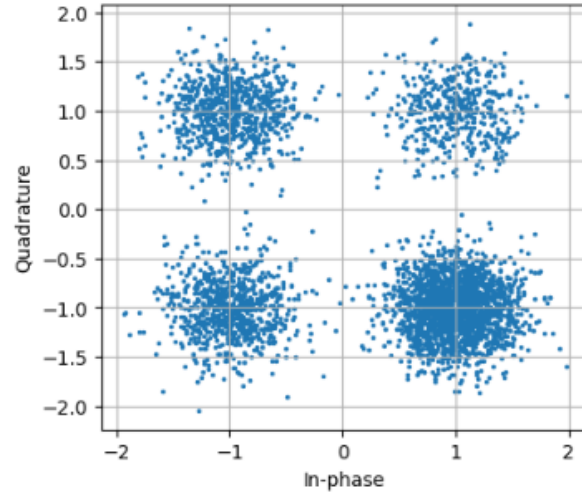
QPSK Scatter Plot ($E_b/N_0 = 6 \text{ dB}$)



$E_b/N_0 = 8 \text{ dB}$, $\text{BER} = 1.945\text{e-}04$



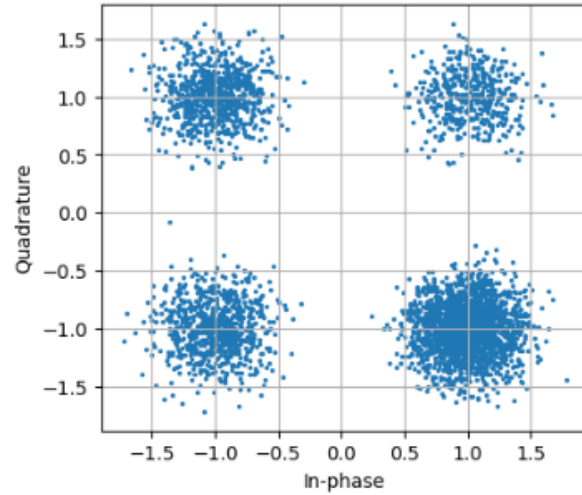
QPSK Scatter Plot ($E_b/N_0 = 8 \text{ dB}$)



$E_b/N_0 = 10 \text{ dB}$, $\text{BER} = 5.722\text{e-}06$



QPSK Scatter Plot ($E_b/N_0 = 10 \text{ dB}$)



BER vs Eb/N0 (Simulated vs Theoretical)

